

RUMP

Reliable UDP Messaging Protocol

PURPOSE

Providing **reliable message delivery over UDP and unreliable communication** channels.

Anyways, I've decided to use UDP.

In harsh network conditions, **TCP may perform poorly due to retransmissions, connection management, and congestion control.**

So, reliability is handled by the application instead.

Here are the features I've implemented to ensure that the data is resilient enough to survive :

- **Fragmentation**
- **Error Detection (CRC)**
- **Forward Error Correction (Reed-Solomon)**
- **Application-layer acknowledgements**

- Fragmentation

- Large messages are divided into fixed-size blocks
- Then, each block is split into multiple data shards, allowing partial recovery.
- This allows me(?) to implement some nice recovery features.

- Error Detection (CRC)

- Each packet includes a CRC32 checksum. The server recalculates the checksum, if it does not match, then, the packet is discarded.

- Forward Error Correction (Reed-Solomon)

- Additional parity shards are generated to allow recovery of lost packets (Using Reed-Solomon algorithm.) without requiring retransmission.

Message header

- Unsigned 32-bit integer message id
- Unsigned 32-bit integer message length
-
- Unsigned 32-bit integer block count
- Unsigned 32-bit integer block id
-
- Unsigned 16-bit integer Total shards
- Unsigned 16-bit integer Amount of data shards
- Unsigned 16-bit integer Amount of parity shards
-
- Unsigned 16-bit integer shard id
- Unsigned 16-bit integer payload size
-
- Unsigned 32-bit integer crc32 checksum
-
- Unsigned char data[SHARD_SIZE]

Server's response header

- Unsigned 32-bit integer Message id
- Unsigned 32-bit integer Status
- Unsigned 32-bit integer crc32;

Pipeline

